

Constrained EML Grammars in Symbolic Regression: Structural Inflation and Throughput Loss

Biswajyoti Nath

Independent Study, <https://github.com/biswajyoti-nath/eml-framework>

Abstract. In evolutionary symbolic regression, restricting the operator space changes the distribution of valid candidates, but the cost of strict validity enforcement is often hidden by nominal budget accounting. This paper presents a controlled empirical study of how a constrained operator grammar affects symbolic search behavior under strict domain constraints. We analyze a grammar built from a single nonlinear primitive, $\text{eml}(x, y) = \exp(x) - \ln(y)$, under strict real-domain positivity constraints using grammar-pure mutation and rejection-aware budget accounting. Across 18 domain-safe targets, recursive EML macro-expansion increased symbolic tree depth (4.4 ± 0.9 vs 3.2 ± 0.9 baseline). This inflation reduced the fraction of valid candidates, raising aggregate rejection rates to 85.9%. Identical nominal evaluation budgets therefore yielded $\approx 67\%$ fewer productive generations. Depth-triggered rejections accounted for 75.2% of all failures under EML, indicating that structural overflow, not numerical instability, is the primary constraint. These results link rewrite rules to effective evolutionary throughput and show that rejection-aware accounting is necessary for evaluating constrained symbolic systems.

Keywords: symbolic regression · operator-space constraints · structural inflation · rejection rate · EML representation

1 Introduction

In symbolic regression, the choice of grammar determines which expressions are directly reachable, which require multi-step encoding, and which are excluded by domain constraints. Restricting the operator set changes the density of valid candidates, the depth of minimal encodings, and the fraction of the evaluation budget consumed by rejected individuals. Highly constrained grammars therefore provide a controlled setting for measuring how operator-space restrictions affect valid search regions under strict validity enforcement.

This work studies one such restriction: replacing the standard exp/log operator set with a single nonlinear primitive, the EML operator,

$$\text{eml}(x, y) = \exp(x) - \ln(y), \tag{1}$$

which has been shown to generate all elementary functions over the complex field when composed with the constant 1 [1]. Our scope is narrower and empirical. We ask: when this operator restriction is enforced in a real-valued system—where logarithm arguments must be strictly positive, complex-field identities are unavailable, and every rejected candidate consumes evaluation budget—how does the resulting search behavior differ from that of the standard grammar?

We make no claims regarding universal expressivity or algorithmic superiority. The goal is to isolate the effects of operator restriction on search structure and throughput. We contribute: (1) an operational definition of the real-domain EML fragment; (2) a reproducible experimental design with rejection-aware budget accounting that separates domain, depth, and numeric failures; and (3) empirical evidence that structural inflation and throughput reduction follow from this operator restriction.

The remainder of the paper proceeds as follows. Section 2 defines the EML representation and its domain constraints. Section 3 describes the experimental design. Sections 4 and 5 report structural and regression results, respectively. Section 6 discusses limitations and implications.

2 Background and Representation

2.1 Related Work in Constrained Search

The interaction between grammar constraints and search structure has been studied in genetic programming. Montana’s Strongly Typed GP [6] showed that restricting the operator space via type constraints reduces the search space, though it often requires complex initialization to find valid trees. Grammar-guided GP [7,9] extended these ideas by using formal grammars to constrain program structure, showing that production rules determine the topology of the candidate space. Work in constrained symbolic regression [8] and semantic constraints [10] has examined how domain restrictions change the distribution of valid candidates. Operator-restricted search has also been studied in the context of parsimony and interpretability [11]. This paper extends these results by measuring the throughput cost of strict domain constraints under a single-operator grammar, with explicit rejection-type accounting.

2.2 The EML Fragment

The representational choices described in this section determine both the structural properties of EML expressions and the domain constraints that govern candidate validity in the search experiments that follow.

The EML operator is a binary primitive with mixed exponential-logarithmic semantics. Under real-valued computation, the second argument of $\text{eml}(x, y)$ must remain strictly positive so that $\ln(y)$ is well defined. The implementation treats the following identities as the basis for EML encoding:

$$\exp(x) = \text{eml}(x, 1), \tag{2}$$

$$\ln(x) = 1 - \text{eml}(0, x), \tag{3}$$

where the second identity requires $x > 0$ in the real domain. These rewrite rules are the source of the structural inflation analyzed in later sections: each unary transcendental node expands into a multi-node EML subexpression. Because arithmetic absorption requires recursive macro-expansion, structural growth increases relative to native exp/log representations.

More complex operations are absorbed into the EML primitive whenever positive-domain assumptions permit. Multiplication can be encoded as

$$a \cdot b = \exp(\ln(a) + \ln(b)), \quad (4)$$

and non-integer powers as

$$x^a = \exp(a \cdot \ln(x)) \quad (x > 0). \quad (5)$$

Division can be encoded as

$$x/y = \exp(\ln(x) - \ln(y)), \quad (6)$$

which expands to $\text{eml}(\ln(x) - \ln(y), 1)$ under EML. Square roots follow as

$$\sqrt{x} = \exp\left(\frac{1}{2} \cdot \ln(x)\right) \quad (x > 0), \quad (7)$$

and more complex operations compound this inflation. For instance, the expression $\sqrt{x}/(x+1)$ requires encoding as $\exp(\frac{1}{2} \ln(x) - \ln(x+1))$, which in EML form becomes $\text{eml}(\frac{1}{2} \ln(x) - \ln(x+1), 1)$, where each $\ln$ term itself expands into $1 - \text{eml}(0, \cdot)$. Even simple algebraic operations thus produce deeply nested EML structures: a compact expression in standard notation becomes a multi-level tree under the constrained grammar.

Symbolic positivity checking is conservative by design. The domain tests return false unless SymPy can prove positivity under its assumption system, so symbolic certifiability is a stricter condition than mathematical positivity on a restricted real interval. This conservatism increases rejection density but ensures that every accepted candidate is semantically valid.

2.3 Structural Inflation and Grammar Purity

Structural inflation refers to the increase in node count and tree depth that results from rewriting expressions into the EML fragment. This is not an implementation artifact; it follows from encoding multiple nonlinear operators through a single binary primitive. Each $\exp(x)$ node expands into $\text{eml}(x, 1)$, and each $\ln(x)$ becomes $1 - \text{eml}(0, x)$, adding nesting and auxiliary nodes for each transcendental subexpression.

Because arithmetic absorption requires recursive macro-expansion, structural growth increases relative to native exp/log representations. Unary transcendental operators expand into multi-node EML subexpressions, and compositions cascade this expansion. For example, a simple product $a \cdot b$ requires encoding as $\exp(\ln(a) + \ln(b))$, which in EML form becomes $\text{eml}(\ln(a) + \ln(b), 1)$, where each

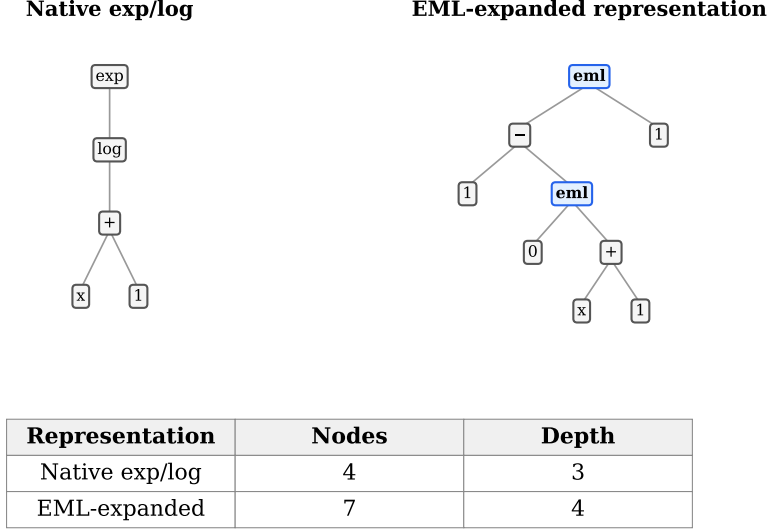


Fig. 1: Side-by-side comparison of $\exp(\ln(x + 1))$ under native exp/log representation (left) and EML-expanded representation (right). The EML encoding replaces native nonlinear composition with recursive primitive expansion. This inflation emerges directly from the rewrite rules and scales with expression complexity.

$\ln$ term itself expands. Expressions with multiplicative structure or nested transcendental compositions therefore experience higher inflation. Figure 1 illustrates this expansion for the expression $\exp(\ln(x + 1))$.

To isolate the effect of operator restriction, the implementation enforces *grammar purity*: EML mutation operates strictly within the EML grammar, with `eml` as the sole nonlinear primitive and all multiplication and power operations expressed via EML identities. The baseline grammar uses native SymPy operators (`exp`, `ln`, multiplication, integer power) without restriction.

2.4 Rejection Dynamics and Evaluation Budget

The study adopts a strict operational definition of budget: **every attempted candidate consumes one evaluation**, whether it is discarded by a depth gate, rejected for domain invalidity, or accepted for fitness scoring. This true budget accounting prevents the analysis from masking invalid sampling behind a nominal budget cap.

Rejection rate is treated as a first-class characterization metric, not a nuisance variable to normalize away. The causal chain is direct: higher rejection density indicates sparser valid regions within the sampled grammar; sparser valid regions mean that a larger fraction of the evaluation budget is consumed by non-productive candidates; and this in turn reduces effective search throughput—the

number of productive generations the search can complete under a fixed budget. This relationship between rejection density and effective throughput is central to the results that follow.

3 Experimental Design

This work is structured as a controlled empirical study to characterize how constrained grammars affect search dynamics. We define four independent variables: the grammar type (EML versus native exp/log), the maximum tree depth limit, the validity enforcement policy (strict symbolic positivity), and the fixed evaluation budget. The dependent variables measured are symbolic node count, tree depth, rejection rate, rejection decomposition (domain, depth, and numeric), completed generations, and mean squared error (MSE).

The structural differences identified in Section 2 motivate three explicit hypotheses:

- **H1:** Recursive EML macro-expansion increases symbolic tree depth and node count relative to native exp/log representations.
- **H2:** Structural inflation under constrained grammars increases rejection density under strict validity enforcement.
- **H3:** Elevated rejection density reduces effective evolutionary throughput under fixed evaluation budgets.

The search procedure is simple to isolate representational effects from optimizer-specific heuristics. Advanced archiving, age-fitness Pareto optimization, and repair mechanisms are excluded so that observed differences are attributable to the grammar rather than the optimizer. The search runs with the following fixed parameters:

- Population size: 100.
- Elite size: 15.
- Total evaluation budget: 4,000 candidate evaluations per run.
- Maximum tree depth: 6.
- Trials: 5 independent runs with deterministic seeds.

The depth cap preserves nominal parity between grammars while exposing the additional structural cost of EML macro-expansion. Every rejected evaluation reduces the remaining budget for productive search.

For each grammar, the search proceeds as follows:

1. Generate an initial valid population via rejection sampling.
2. Score the population with strict numeric validity checks.
3. Retain elites and generate offspring via grammar-pure mutation.
4. Reject any candidate whose actual tree depth exceeds the depth limit.
5. Count every candidate attempt—accepted or rejected—toward the evaluation budget.

The EML grammar uses a dedicated random generator and mutation operators that operate strictly within the EML fragment. The baseline grammar uses standard SymPy tree construction. There are no repair mechanisms, no fallback fitness assignments, and no hidden equivalence testing.

Table 1: Structural inflation and rejection dynamics across 18 targets. Values show mean $\pm$ std across 5 independent trials per target. Rejection rates (%) are disaggregated by type: domain (symbolic constraint violations), depth (structural limits), numeric (evaluation failures), and total. Total rejection is shown as an aggregate percentage and is not accompanied by a standard deviation because it is derived from the category-level totals. Depth rejection dominates under EML ($75.2 \pm 0.7\%$ vs. $6.6 \pm 6.0\%$ baseline) with notably lower variance, indicating consistent structural capacity constraints across targets.

Grammar	Nodes	Depth	Domain (%)	Depth (%)	Numeric (%)	Total (%)	Gens
exp/log baseline	10.6 ± 4.0	3.2 ± 0.9	6.5 ± 5.0	6.6 ± 6.0	31.5 ± 9.7	44.6	15.5 ± 1.0
EML grammar	14.8 ± 5.2	4.4 ± 0.9	0.6 ± 0.8	75.2 ± 0.7	10.0 ± 1.5	85.9	5.1 ± 0.4
Ratio (EML/bl)	$1.4\times$	$1.4\times$	$0.1\times$	$11.4\times$	$0.3\times$	$1.9\times$	$0.3\times$

Rejection is not treated as a monolithic failure. To clarify the causal mechanics of search distortion, we explicitly define and separate three rejection types:

- **Domain Rejection:** Candidates rejected during symbolic verification because their structure violates mathematical domain constraints (e.g., negative arguments to a logarithm).
- **Depth Rejection:** Candidates rejected because their structure exceeds the strict maximum depth cap.
- **Numeric Rejection:** Candidates rejected during fitness evaluation due to numerical overflow, NaN generation, or floating-point errors on the sample grid.

This separation prevents the conflation of syntactic depth limits with semantic domain violations, allowing future iterations of this study to report detailed rejection profiles for each grammar.

The regression targets comprise 18 domain-safe functions selected from a larger expression catalogue. Each target is evaluated on a fixed sample grid of 201 points within a positive-domain interval. All experiments are deterministic given the seed settings in `src/eml/config.py`.

4 Structural Characterization Results

The structural analysis quantifies the representational cost of operator restriction and tests our hypotheses. Table 1 aggregates the key metrics across all 18 targets.

Consistent with **H1**, the EML grammar increased expression size, producing trees with 14.8 ± 5.2 nodes and 4.4 ± 0.9 depth on average, compared to 10.6 ± 4.0 nodes and 3.2 ± 0.9 depth for the baseline ($1.4\times$ inflation). This increase is a direct consequence of recursive macro-expansion.

Supporting **H2**, this structural inflation raised rejection density. EML experienced $1.9\times$ higher aggregate rejection rates (85.9%) than the baseline (44.6%). Depth-triggered rejections reached $75.2 \pm 0.7\%$ under EML (compared to $6.6 \pm 6.0\%$ in the baseline). The deep encodings required for domain safety exhaust the

depth budget before semantic verification occurs, reducing the fraction of valid candidates.

The distribution of rejection rates across targets further illustrates the search-space sparsity difference between grammars. Figure 2 shows that EML rejection rates cluster tightly between 71% and 79%, while baseline rates span a wider range from 18% to 45%. This indicates that the valid candidate region under EML is not only sparser on average but also consistently sparse across diverse target structures.

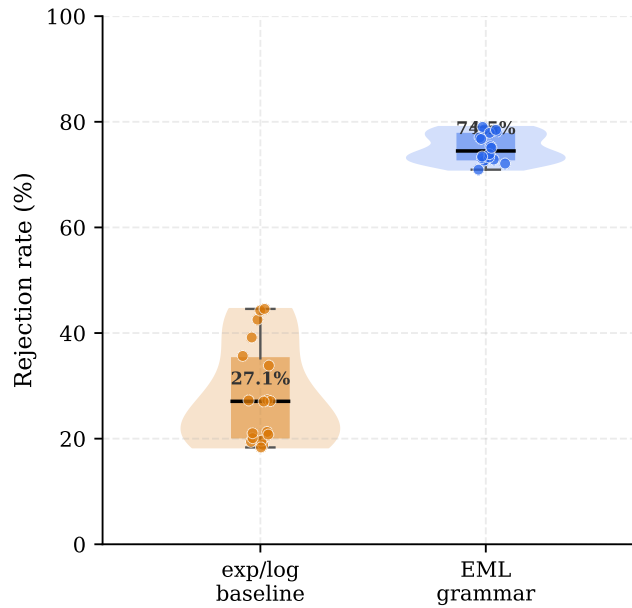


Fig. 2: Boxplot comparison of aggregate rejection rates across 18 targets under EML and baseline grammars. EML exhibits consistently higher rejection rates (median approximately 75%) compared to the baseline (median approximately 27%). These aggregate rates include domain, depth, and numeric rejections under strict validity enforcement.

The mechanism underlying these differences is traceable to the rewrite rules. Each $\exp(x)$ becomes $\text{eml}(x, 1)$ and each $\ln(x)$ becomes $1 - \text{eml}(0, x)$, adding nesting and auxiliary nodes for every transcendental subexpression. Multiplicative encodings such as $a \cdot b = \exp(\ln(a) + \ln(b))$ compound this effect when the operands are themselves nontrivial. Structural inflation is therefore not an artifact to be normalized away; it is an inherent representational cost of the constrained grammar.

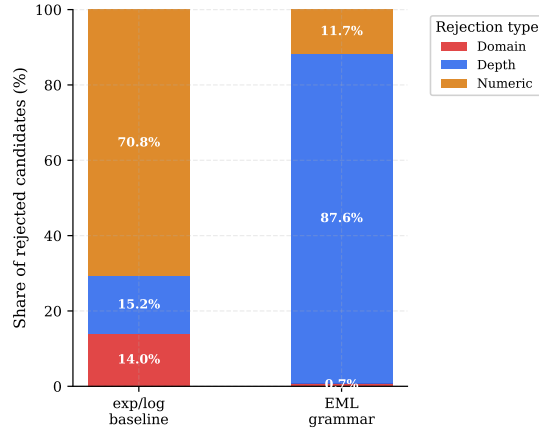


Fig. 3: Composition of rejected candidates by type. Percentages are expressed relative to the total number of rejected candidates, so the bars sum to 100% within each grammar. Under EML, depth-triggered rejections constitute 87.6% of all rejections, reflecting structural inflation as the dominant throughput constraint. Domain rejections comprise only 0.7% of EML rejections, while numeric rejections account for 11.7%. The baseline grammar shows a more balanced composition: numeric rejections (70.8%) predominate, with depth (15.2%) and domain (14.0%) rejections contributing more equally.

The disaggregated rejection metrics reveal that depth rejection dominates under EML ($75.2 \pm 0.7\%$ vs. $6.6 \pm 6.0\%$ baseline), as structural inflation exhausts the depth cap before domain verification. Domain and numeric rejections are correspondingly lower. Figure 3 shows EML rejections are 87.6% depth-related, versus a more balanced distribution in the baseline.

5 Symbolic Regression Results

The structural inflation and rejection penalties documented above directly limit the evolutionary process. In support of **H3**, the elevated rejection density under EML reduced effective evolutionary throughput. Despite identical nominal budgets of 4,000 evaluations, EML completed only 5.1 ± 0.4 productive generations on average, compared to 15.5 ± 1.0 for the baseline. This $\approx 67\%$ reduction in throughput shows that constrained validity enforcement carries a substantial operational cost.

The mechanism linking rejection to throughput is visualized in Figure 4, which plots rejection rate against completed generations for each target under both grammars.

EML targets cluster in the high-rejection, low-generation region, while baseline targets occupy the opposite quadrant. Higher rejection density directly reduces the evaluation budget available for productive evolutionary updates.

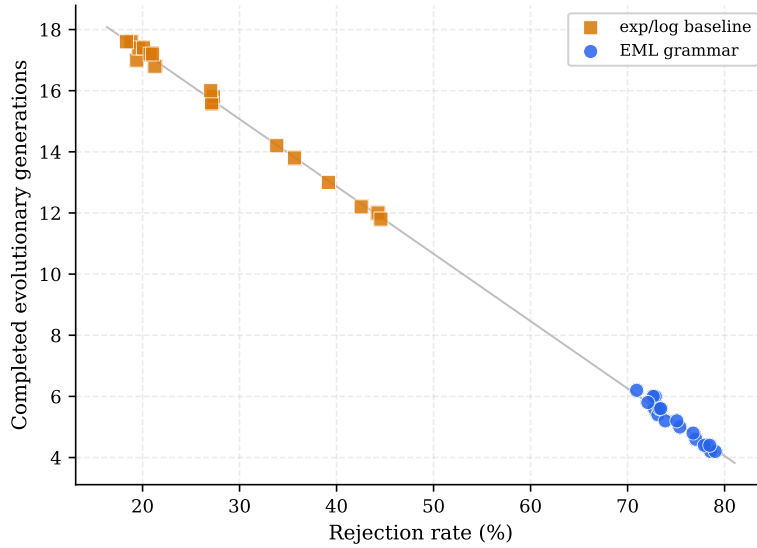


Fig. 4: Operational consequence of rejection-aware budget accounting: higher aggregate rejection density reduces the number of productive generations achievable under a fixed evaluation budget of 4,000 candidates. This clustering pattern is an expected outcome of the experimental design—as rejection rates increase, proportionally fewer evaluations remain for productive evolutionary updates—not a claim of discovering a novel statistical law.

The exploratory regression experiment produced additional characterization observations:

- Below-threshold MSE (indicating functional recovery) was observed on 8 of 18 targets under the EML grammar and 6 of 18 under the baseline. These counts are reported as search-behavior indicators, not as competitive performance metrics.

These observations characterize how each grammar interacts with the search process but do not show that either grammar is categorically preferable. EML recovered certain targets despite its throughput penalty. Functions with recursive exp/log structure—such as pure exponentials ($\exp(x)$, $\exp(x^2)$) or nested compositions ($\exp(\ln(x + 1))$)—were reliably found because these structures map directly onto the EML primitive’s encoding pattern. The baseline grammar failed to recover several of these same targets (e.g., $\exp(x^2)$, $\exp(x) + x$) within the allotted budget. Under strict computational constraints, functional recovery depends on structural compatibility with the grammar more than on operator diversity. Conversely, mixed multiplicative targets ($x^2 \cdot \ln(x)$, $x^2 \cdot \exp(x)$) and rational forms ($x/(1 + x)$) required deeper macro-expansions that compounded rejection penalties under EML. Recoverability thus varies with the structural

affinity between the target and the grammar’s encoding rules, and both grammars exhibit target-dependent recoverability profiles.

The central observation is that equal nominal budgets do not imply equal effective search capacity. The same 4,000 candidate evaluations yield fewer productive generations for EML because a larger fraction of the budget is consumed by rejected candidates and by the deeper structures that surviving candidates require.

6 Discussion and Limitations

The results show three interrelated consequences of operator restriction.

Conservative Validity Enforcement The EML transformer rejects expressions unless SymPy can prove all logarithm arguments and non-integer power bases are positive. Symbolic certifiability is therefore a stricter condition than mathematical positivity on a restricted domain: expressions that are numerically valid on the sampled interval may still fail the symbolic check. This conservatism increases rejection density but prevents semantically invalid candidates from entering the population.

Operator Restriction Reduces Valid Search-Space Density Condensing multiple nonlinear primitives into a single binary operator reduces the grammar’s vocabulary. However, this reduction also reduces the fraction of candidate expressions that satisfy domain constraints. The high rejection rates under EML are not a temporary algorithmic artifact; they measure the sparsity of the valid region within the constrained candidate space.

Structural Inflation Interacts with Depth Constraints The EML grammar can encode many functions in principle, but doing so requires systematically deeper and wider trees. Under a strict depth cap, this inflation reduces the reachable portion of the search space. The resulting representation-induced search distortion—where structurally deeper encodings consume depth budget that would otherwise support broader exploration—is an inherent property of the constrained grammar, not a quantity to be normalized away.

Rejection Type Analysis Clarifies Operational Mechanisms The disaggregated rejection metrics separate the contributions of each failure mode. Depth rejection dominates under EML ($75.2 \pm 0.7\%$ vs. $6.6 \pm 6.0\%$ baseline), indicating that structural inflation is the primary throughput constraint—the grammar’s encoding of domain-safe representations consumes depth budget at scale. Domain rejection is lower under EML (0.6% vs. 6.5% baseline) because depth filtering occurs first, removing structurally excessive candidates before domain verification. Numeric rejection is also lower under EML (10.0% vs. 31.5%

baseline) as a secondary effect of depth filtering. The ordering is: structural inflation triggers depth rejection, which limits the candidates reaching domain verification. This indicates that EML’s primary limitation is representational compactness under domain constraints, not semantic validity. The constrained grammar produces valid expressions when depth permits; the constraint is that permitted depths are shallower due to macro-expansion.

Limitations This paper characterizes constrained-search behavior under controlled conditions rather than establishing universal symbolic regression performance properties. The benchmark suite is exploratory and restricted to 18 targets expressible within the positive-domain exp–log fragment. The evolutionary algorithm is simple to isolate representation effects, lacking state-of-the-art archiving or age-fitness Pareto optimization. The EML fragment studied here excludes trigonometric, hyperbolic, and complex-valued constructions, so the findings apply strictly to the real-domain positivity-constrained subset. Conservative symbolic positivity checking may also reject expressions that are numerically safe under more sophisticated interval analyses.

Implications These results suggest that validity-aware budget accounting is useful for analyzing effective search capacity in constrained symbolic systems. When invalid candidate generation consumes nontrivial computational resources, evaluating grammars on nominal budgets alone misrepresents the operational cost. The link between structural inflation and rejection density provides a basis for designing operator restrictions and adaptive depth constraints in domain-aware symbolic regression.

7 Conclusion

This study measured how a constrained operator grammar affects validity-aware symbolic search under fixed evaluation budgets. Restricting the nonlinear operator set to $\text{eml}(x, y)$ increased tree depth (4.4 ± 0.9 vs 3.2 ± 0.9 baseline). This inflation increased rejection density (85.9% vs 44.6% baseline), primarily through depth-limit overflows. The resulting reduction in valid candidates yielded $\approx 67\%$ fewer productive generations under identical nominal evaluation budgets.

These three effects are coupled: recursive macro-expansion increased the structural footprint of candidate solutions, which increased rejection density under strict validity enforcement, which reduced the number of productive generations under fixed evaluation budgets. These claims are bounded by the conditions of our study: an 18-target suite on a positive-domain fragment, using a simple evolutionary algorithm without advanced archiving. The results therefore characterize the structural distortion induced by the constrained grammar, not its peak competitive potential. Validity-aware accounting may apply to other constrained symbolic systems as a more transparent metric for evaluating representational costs.

References

1. Odrzywołek, A.: All elementary functions from a single binary operator. arXiv preprint arXiv:2603.21852 (2026)
2. Udrescu, S.M., Tegmark, M.: AI Feynman: A physics-inspired method for symbolic regression. *Science Advances* **5**(3) (2019)
3. Petersen, K., Zhou, J., Nelatury, A., Carmel, D.: Deep symbolic regression. In: *Proceedings of the International Conference on Learning Representations* (2021)
4. La Cava, W., Moore, J.H., Urbanowicz, R.J.: FEAT: Feature engineering automation tool for predictive modeling. *Journal of Machine Learning Research* **21**(150) (2020)
5. Schmidt, M., Lipson, H.: Distilling free-form natural laws from experimental data. *Science* **324**(5923), 81–85 (2009)
6. Montana, D.J.: Strongly typed genetic programming. *Evolutionary Computation* **3**(2), 199–230 (1995)
7. McKay, R.I., Hoai, C., Whigham, P.M., Shan, Y., O’Neill, M.: Grammar-based genetic programming: a survey. *Genetic Programming and Evolvable Machines* **11**, 365–396 (2010)
8. Lu, Q.: Constrained symbolic regression. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* **46**(10), 1345–1356 (2016)
9. Whigham, P.A.: Grammatically-based genetic programming. In: *Proceedings of the Workshop on Genetic Programming: From Theory to Real-World Applications*, pp. 33–41 (1995)
10. Trujillo, L., Munoz, A., Schoenauer, L., Secretan, M.: Optimization for the evolving grammar-based genetic programming. *Genetic Programming and Evolvable Machines* **17**, 483–513 (2016)
11. Virgolin, M., Aldén, A., Alderliesten, T.: Learning where to look: Neural attention for symbolic regression. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pp. 1635–1643 (2021)